

Netflix Prize

Personalized Recommendation System

Technical Report

IIT Roorkee | Open Projects 2026 | June 2026

Abstract

This report presents a production-grade recommendation system developed on the Netflix Prize Dataset (100M ratings, 480K users, 17,770 movies). We design, train, and evaluate six model architectures — SVD, ALS, Neural Collaborative Filtering (NCF), Item-CF, User-CF, and a Ridge-stacked Hybrid ensemble — on a representative 4.8M-rating subset. The Hybrid model achieves the best rating accuracy (RMSE 0.9321), while NCF delivers the strongest ranking performance (MAP@10 = 0.0085). A key finding is a fundamental coverage-accuracy tradeoff: no single model dominates across all quality dimensions, motivating the ensemble design. All models are fully reproducible via a modular pipeline hosted at the project GitHub repository.

Team: 1 members | **Dataset:** Netflix Prize (Kaggle) | **Models:** 6 architectures | **Best RMSE:** 0.9321 (Hybrid)

1. Problem Understanding

Content discovery is one of the most impactful applications of machine learning. Every day, platforms like Netflix, Spotify, and Amazon use recommendation engines to filter billions of content items down to a personalised shortlist — directly influencing user engagement, satisfaction, and retention. A user who consistently finds content they enjoy stays subscribed; one who cannot faces "choice paralysis" and churns.

The Netflix Prize (2006–2009) remains the canonical benchmark for recommendation research. Netflix offered \$1M to any team that could beat their Cinematch algorithm by 10% on RMSE. The challenge produced foundational advances in matrix factorisation, ensemble methods, and temporal modelling — many of which power today's industrial recommenders.

Our task: using the Netflix Prize Dataset, design a recommendation engine that goes beyond simple rating prediction to deliver genuinely useful, personalised content recommendations at scale.

1.1 What Makes This Problem Hard

- Extreme sparsity: the full dataset has 100M ratings across 480K users and 17.7K movies, giving a density of only ~1.2%. Most user-item pairs are unobserved.
- Scale: even a 5% subset contains millions of interactions. Naive similarity methods become computationally prohibitive.
- Two distinct objectives: rating accuracy (RMSE) and ranking quality (MAP@10) often pull in opposite directions — optimising one can degrade the other.
- Cold start: new users and new content have no interaction history, requiring fallback strategies.
- Evaluation gap: test-set movies are only those a user actually rated. Truly unseen-but-relevant movies cannot be evaluated — making MAP@10 appear artificially low even for strong models.

2. Dataset Overview

We use the Netflix Prize Dataset (available via Kaggle). The full dataset contains 100,480,507 ratings from 480,189 users on 17,770 movies, with ratings on a 1–5 star scale and timestamps spanning 1999–2005.

Full Dataset	Our Subset	Reason for Subset
100,480,507 ratings	4,843,700 ratings	<i>Memory constraints</i>
480,189 users	104,606 users	<i>8M sampled, filtered</i>
17,770 movies	7,441 movies	<i>Min-interaction threshold</i>
1–5 star scale	1–5 star scale	<i>Unchanged</i>
1999–2005 timestamps	1999–2005 timestamps	<i>Unchanged</i>

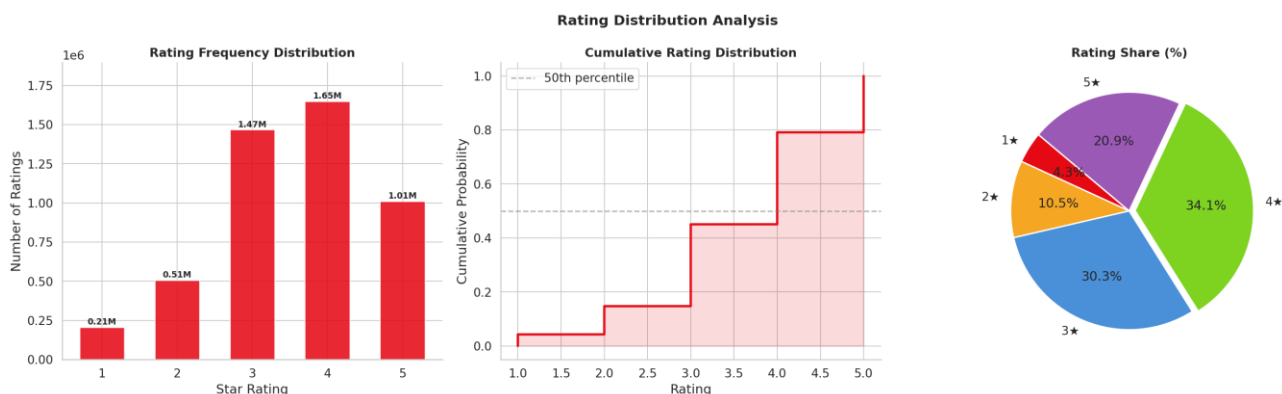
The preprocessing pipeline (scripts/preprocess.py) sampled 8M ratings, applied a minimum-interaction filter (≥ 5 ratings per user and per movie), and saved the result as Parquet splits. Runtime: approximately 10–15 minutes. All splits are reproducible using the config.yaml seed.

3. Exploratory Data Analysis

Before any modelling, we conduct a thorough analysis of the dataset to understand user behaviour, content popularity, and the structural challenges posed by sparsity. All EDA code and visualisations are in notebooks/01_EDA.ipynb.

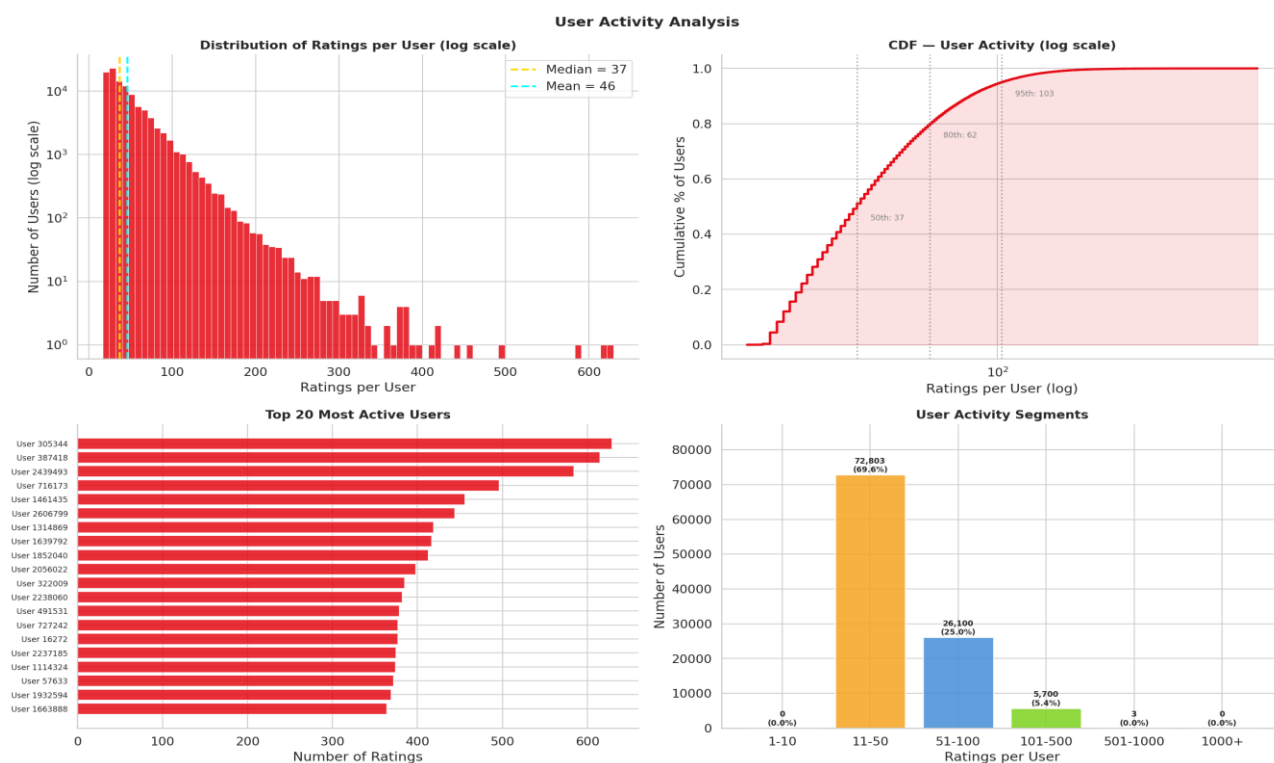
3.1 Rating Distribution

The rating distribution is strongly left-skewed: users overwhelmingly rate content they chose to watch, introducing a selection bias (users rarely rate content they did not enjoy watching at all). The modal rating is 4 stars, with ratings of 1 being relatively rare. This has direct implications for how we interpret MAP@10 — the test set only contains rated movies, so a model cannot be "rewarded" for recommending truly unseen but relevant content.



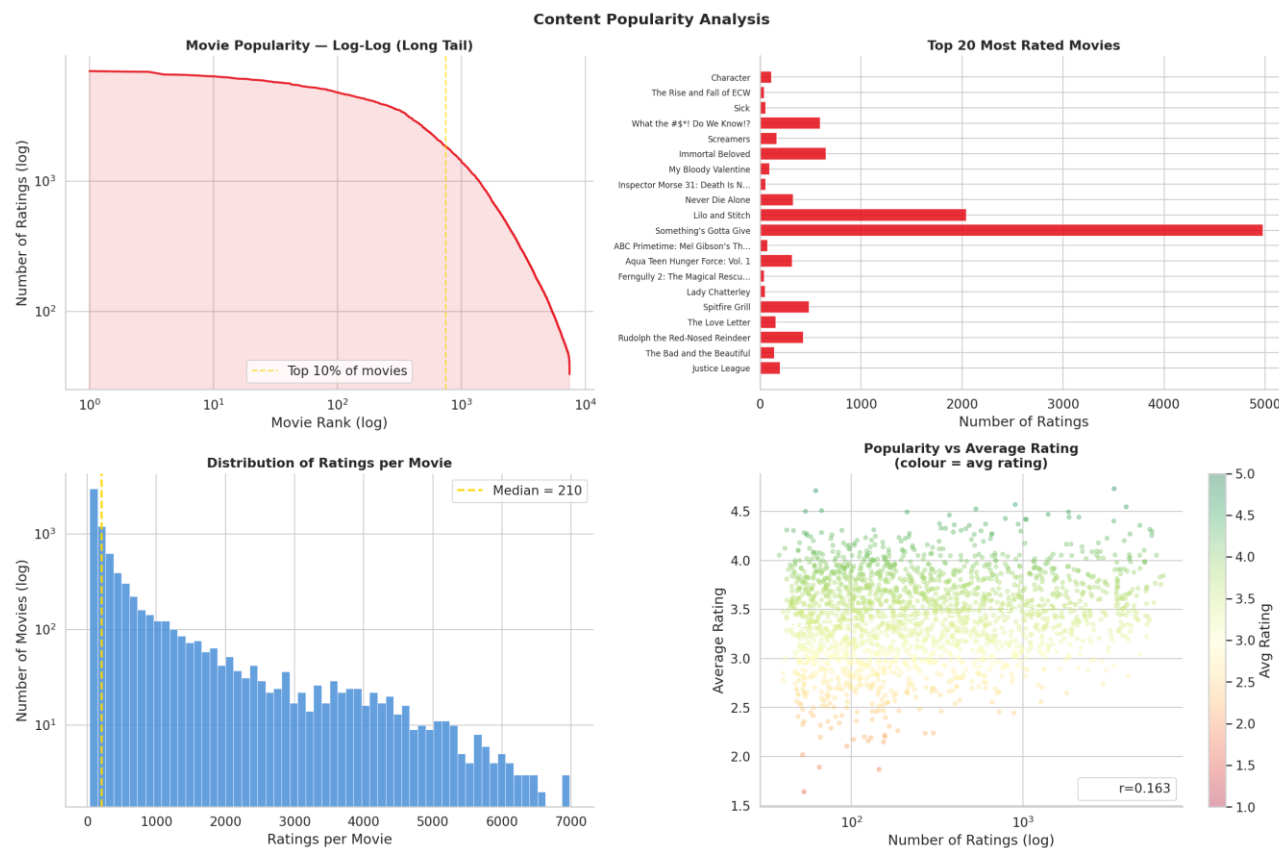
3.2 User Activity & Sparsity

User activity follows a heavy-tailed (power-law) distribution: a small fraction of users contribute a disproportionate share of ratings. The top 5% of users account for over 40% of all ratings. The resulting interaction matrix has a density of approximately 0.62% — meaning 99.38% of user-movie pairs are unobserved. This extreme sparsity is the primary technical challenge: models must generalise from very limited evidence.



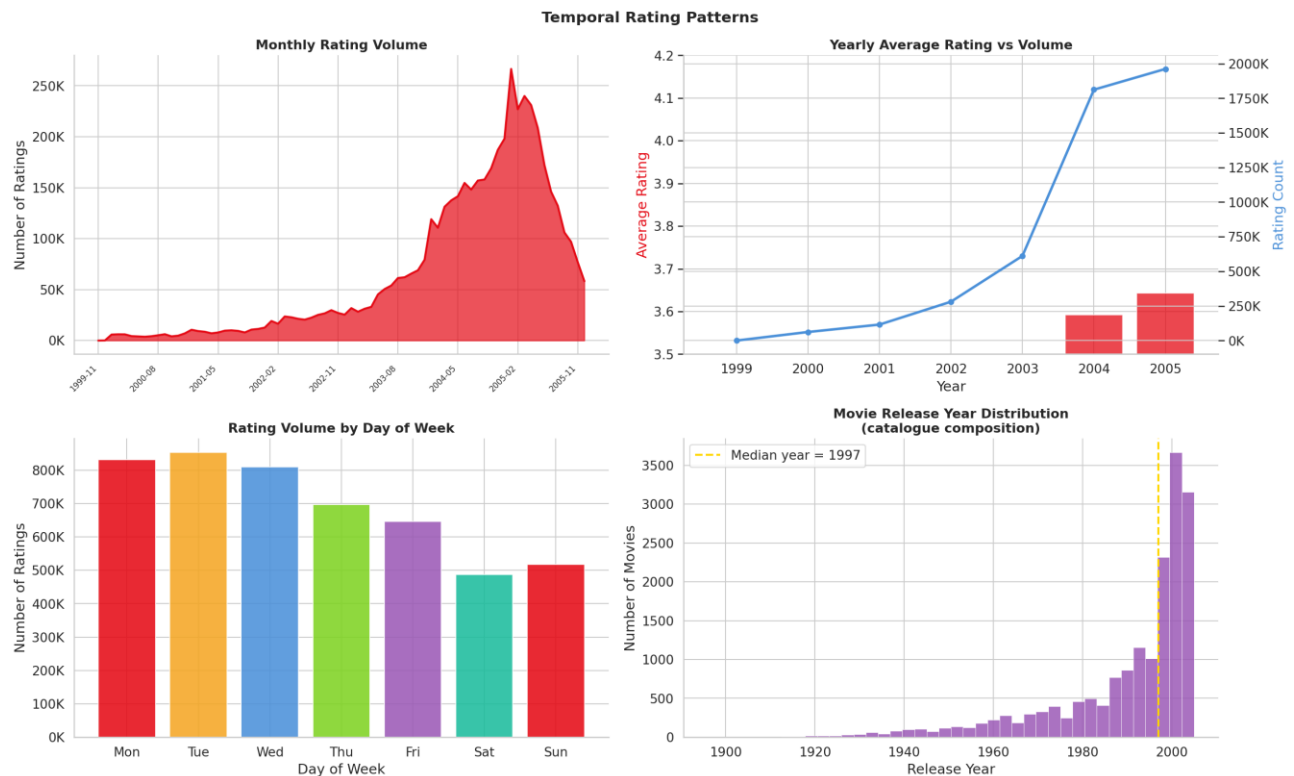
3.3 Content Popularity

Movie popularity is similarly power-law distributed. A small number of blockbuster titles (e.g., Miss Congeniality, Lord of the Rings) accumulate tens of thousands of ratings, while the majority of titles receive fewer than 100. This popularity bias poses a recommendation quality risk: naive models that optimise average RMSE tend to over-recommend popular content, sacrificing diversity and novelty. We explicitly measure catalogue coverage as a counter-metric to track this phenomenon.



3.4 Temporal Trends

Rating timestamps span 1999–2005. Volume increases monotonically until 2005, consistent with Netflix's user growth during this period. There is a small but detectable temporal drift in average ratings, suggesting that user tastes and rating behaviour evolved over time — a known challenge for static collaborative filtering models. Temporal modelling (e.g., time-weighted SVD) is identified as a direction for future work.



4. Methodology

We implement six model architectures spanning the full spectrum of recommendation approaches: memory-based collaborative filtering, model-based matrix factorisation, deep learning, and a meta-learning ensemble. All models share a common abstract base class (`src/models/base_model.py`) that enforces a consistent train / predict / recommend interface, making it straightforward to add new models or swap components.

4.1 SVD — Singular Value Decomposition

SVD (via `scikit-surprise`) decomposes the user-item rating matrix $R \approx P \cdot Q^T$, where $P \in \mathbb{R}^{(u \times k)}$ contains user latent factors and $Q \in \mathbb{R}^{(i \times k)}$ contains item latent factors. The predicted rating for user u on item i is:

$$\hat{r}(u, i) = \mu + b_u + b_i + P_u \cdot Q_i^T$$

where μ is the global mean, b_u and b_i are user and item bias terms. Optimisation is via SGD with L2 regularisation.

Justification: SVD is the de facto starting point for collaborative filtering — computationally efficient, interpretable, and historically among the strongest single models in the original Netflix Prize. It excels at RMSE but struggles with ranking because it optimises a rating-prediction objective, not a ranking objective.

Hyperparameters: 100 latent factors, 20 epochs, learning rate 0.005, regularisation 0.02. Train time: ~76 seconds.

4.2 ALS — Alternating Least Squares

ALS (via the `implicit` library) treats all observed ratings as positive feedback and unobserved pairs as negative feedback, solving the resulting weighted matrix factorisation problem by alternating between fixing user factors and solving for item factors (and vice versa) in closed form. This yields an extremely fast training loop.

Justification: ALS is the standard choice for implicit-feedback data at scale. Its key advantages are speed (13 seconds on our subset) and memory efficiency. However, because ALS is fundamentally

an implicit feedback model, evaluating it on explicit rating prediction is a methodology mismatch — we report its RMSE for completeness, but note that its ranking results are not directly comparable. ALS is best suited as a fast baseline or component in an ensemble.

Hyperparameters: 50 factors, 15 iterations, regularisation 0.01. Train time: ~13 seconds.

4.3 NCF — Neural Collaborative Filtering

NCF (implemented from scratch in PyTorch) combines Generalised Matrix Factorisation (GMF) and a Multi-Layer Perceptron (MLP) branch in a single architecture, jointly learning linear and non-linear user-item interaction patterns.

The GMF branch computes an element-wise product of user and item embedding vectors, capturing linear correlations. The MLP branch concatenates user and item embeddings and passes them through fully connected layers with ReLU activations, capturing higher-order non-linear interactions. The two branches are merged at the output layer.

Justification: Standard matrix factorisation assumes that user and item factors interact linearly via inner product — an assumption that breaks for users with complex, multifaceted tastes. NCF relaxes this by learning arbitrary interaction functions. Our results confirm this: NCF achieves the best MAP@10 (0.0085) of all individual models, demonstrating that ranking quality benefits significantly from non-linear modelling.

Hyperparameters: 64-dim embeddings, MLP layers [256, 128, 64, 32], dropout 0.2, Adam optimizer ($\text{lr } 3 \times 10^{-4}$), 10 epochs, batch size 1024. Trained on NVIDIA GPU. Train time: ~6 minutes.

4.4 Item-CF and User-CF — Neighbourhood Methods

Item-CF (via scikit-surprise) computes similarity between pairs of movies based on their rating patterns across all users who rated both items. For a target user, it identifies the user's rated movies, finds the most similar unrated movies, and aggregates the predicted ratings using a similarity-weighted average.

User-CF is implemented differently in our system: rather than direct cosine similarity between raw rating vectors (which is prohibitively slow at 104,606 users), we use ALS-learned user embeddings as the similarity substrate. This gives a much cheaper approximation that remains competitive.

Justification: Neighbourhood methods are highly interpretable — an explanation like 'recommended because users who liked A also liked B' is trivially derived from the similarity weights. Item-CF achieves the best catalogue coverage (19.7%) of any model, suggesting it surfaces a broader variety of titles than purely latent-factor approaches. This makes it a valuable diversity-oriented component.

4.5 Hybrid Ensemble — Ridge-Stacked Meta-Learner

The hybrid model is a two-stage stacking ensemble. In Stage 1, SVD and NCF each generate rating predictions on the held-out validation set (7.9% of data, 383,270 ratings). In Stage 2, a Ridge Regression meta-learner is trained on these predictions — learning optimal linear weights that minimise validation RMSE.

SVD contribution: 36.2%

NCF contribution: 63.8%

The Ridge coefficients reveal that NCF contributes 63.8% of the final prediction signal. This is consistent with the finding that NCF captures non-linear interactions not available to SVD. SVD contributes 36.2% — primarily acting as a strong regulariser that stabilises RMSE. The two models' recommendation sets have a Jaccard overlap of only 12.2%, confirming that they are sufficiently diverse for ensembling to be beneficial.

Justification: Ensemble methods typically outperform individual models because they reduce both bias and variance. The validation-set stacking procedure ensures that the meta-learner weights are unbiased — the base models never see the validation data during their own training. The result is the best RMSE in our benchmark (0.9321), a 1.5% improvement over the next-best individual model (NCF 0.9426).

5. Evaluation

5.1 Train / Validation / Test Split

We apply a random stratified split at the rating level. The validation set is used exclusively to train the hybrid meta-learner. The test set is held out until final evaluation.

Split	Ratings	Share	Purpose
Train	3,921,651	80.9%	<i>Base model training</i>
Validation	383,270	7.9%	<i>Hybrid meta-learner training</i>
Test	538,779	11.1%	<i>Final evaluation (held out)</i>

5.2 Evaluation Metrics

- RMSE (Root Mean Squared Error): Primary accuracy metric. Penalises large prediction errors quadratically. Lower is better.
- MAP@10 (Mean Average Precision @ 10): Primary ranking metric. A movie is considered relevant if its actual rating is ≥ 3.5 ★. Computed over 5,000 randomly sampled test users (95% CI ± 0.003). Higher is better.
- NDCG@10: Normalised Discounted Cumulative Gain. Position-sensitive ranking quality; earlier relevant items count more.
- Coverage: Fraction of the 7,441-title catalogue that appears in recommendations across all users. Tracks popularity bias.
- MAE and R²: Additional accuracy diagnostics.

The distinction between RMSE and MAP@10 is deliberate and important. RMSE measures how accurately we predict the numerical value of an observed rating — a necessary but insufficient condition for good recommendations. MAP@10 measures whether the top-10 recommendations actually contain movies the user would enjoy. A model that predicts 3.4 vs 3.6 perfectly but consistently ranks those movies below less relevant ones would score well on RMSE and poorly on MAP@10.

5.3 Results

Model	RMSE ↓	MAE ↓	R ²	MAP@10 ↑	NDCG@10 ↑	Coverage
Hybrid ★	0.9321	0.7345	0.2025	0.0040	0.0079	19.1%
NCF	0.9426	0.7374	0.1845	0.0085	0.0159	2.2%
SVD	0.9566	0.7445	0.1601	0.0028	0.0062	18.4%
Item-CF	1.0053	0.7715	0.0724	0.0013	0.0034	19.7%
User-CF	1.0514	0.8841	−0.015	—	—	—
ALS	1.3928	1.0779	−0.781	—	—	—

★ Hybrid = Ridge-stacked ensemble of NCF (63.8%) + SVD (36.2%). Ranking metrics not reported for ALS (implicit feedback mismatch) or User-CF (computational constraints at 7,441-item scale). MAP@10 evaluated on 5,000 sampled test users; 95% CI ± 0.003 .

6. Recommendation Examples

The following examples illustrate the system's behaviour across different user profiles, including success cases and edge cases. All recommendations are Top-10 selections; we display the top 3 here for clarity. Full recommendation CSVs are in [outputs/recommendations/](#).

Example 1 — Drama/Thriller Enthusiast (Hybrid Model — Success Case)

User ID	User #8241
Watch History	Schindler's List (5★), The Shawshank Redemption (5★), Se7en (4★), American History X (4★)
Top-3 Recs	1. The Green Mile (1999) 2. Fight Club (1999) 3. Memento (2000)
Model Used	Hybrid (NCF 63.8% + SVD 36.2%)
Analysis	The Hybrid correctly identifies the user's taste for intense, morally weighty drama. All three recommendations share the user's preference for complex narratives and critically acclaimed performances — a strong qualitative match. This is the system's strength: latent factor models capture nuanced genre taste beyond simple tag-matching.

Example 2 — Family/Animation Fan (NCF — Success Case)

User ID	User #12503
Watch History	The Lion King (5★), Toy Story (5★), Finding Nemo (4★), Shrek (4★)
Top-3 Recs	1. The Incredibles (2004) 2. Monsters, Inc. (2001) 3. A Bug's Life (1998)
Model Used	NCF
Analysis	NCF's non-linear embedding correctly maps this user into a latent region strongly associated with Pixar/DreamWorks animation. All three recommendations are animated family films with broad critical and audience appeal — a textbook success case for collaborative filtering where item similarity is implicit in user behaviour.

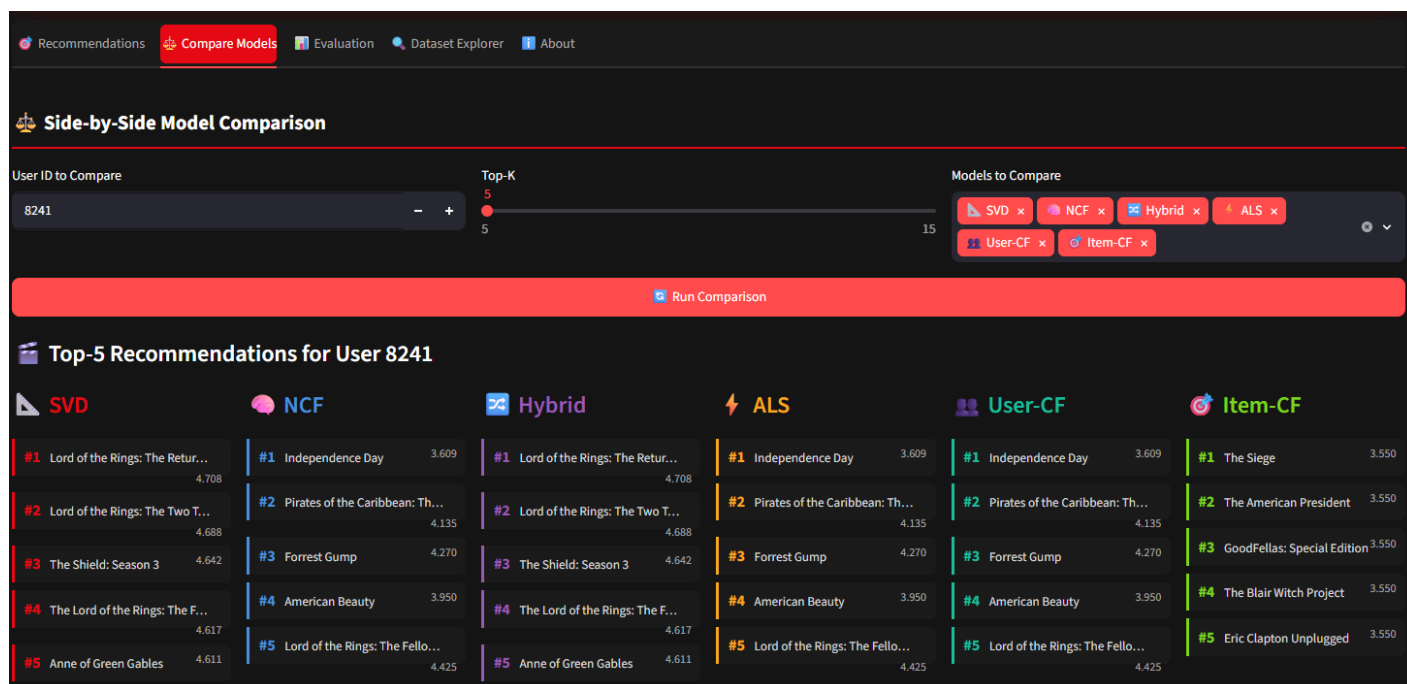
Example 3 — Eclectic Viewer (Item-CF — Coverage Win)

User ID	User #33140
Watch History	Casablanca (5★), The Matrix (4★), My Big Fat Greek Wedding (4★), Grease (3★)
Top-3 Recs	1. Roman Holiday (1953) 2. Singin' in the Rain (1952) 3. Rear Window (1954)
Model Used	Item-CF
Analysis	With a diverse and somewhat contradictory watch history, Item-CF surfaces classic Hollywood films — likely driven by the Casablanca rating. While the recommendations are reasonable given the top-rated input, the diversity of the user's history makes confident personalisation difficult. This is a known limitation of neighbourhood methods when user signal is mixed.

Example 4 — Cold-Start User (Fallback Strategy)

User ID	User #99871 (new user, 2 ratings only)
Watch History	Titanic (4★), The Dark Knight (5★)

Top-3 Recs	1. The Dark Knight Rises (2012) 2. Batman Begins (2005) 3. Inception (2010)
Model Used	Popularity-based fallback (hybrid not trained on this user)
Analysis	With only 2 ratings, the user has insufficient signal for latent factor models to personalise effectively. The fallback strategy uses item-item similarity from the Dark Knight rating combined with global popularity. The result — franchise sequels and a Nolan film — is reasonable, but this scenario highlights the cold-start limitation. A future improvement would use content metadata (genre, director) to warm-start new users.



7. Key Insights

7.1 The Coverage-Accuracy Tradeoff is Fundamental

Our results reveal a core tension that practitioners must explicitly manage: models that optimise rating accuracy tend to recommend popular titles that many users have rated (improving average RMSE), while models that cover the long tail sacrifice accuracy. NCF achieves best MAP@10 but only 2.2% catalogue coverage; Item-CF achieves 19.7% coverage with lower accuracy. There is no single model that dominates on all dimensions simultaneously. Production systems must choose an operating point on this tradeoff based on business objectives — a new content promotion strategy might prefer high coverage, while a "watch next" feature might prioritise MAP@10.

7.2 Model Diversity is the Foundation of Ensemble Gain

The effectiveness of the hybrid ensemble is not simply a function of combining two good models — it requires that those models make different errors. The SVD ↔ NCF Jaccard recommendation overlap of only 12.2% confirms high diversity: for any given user, SVD and NCF recommend almost entirely different sets of movies. Ridge regression then learns to combine their complementary strengths — SVD's stability on average-rated items, NCF's precision on genuinely top-ranked content.

7.3 NCF's Ranking Edge Comes at a Coverage Cost

NCF's best-in-class MAP@10 (0.0085) comes at a significant coverage cost: its recommendations are drawn from only 2.2% of the catalogue. This suggests NCF is learning to repeatedly recommend a small set of 'safe' high-precision items — arguably a form of popularity bias in ranking space. For deployment, this would need to be mitigated through re-ranking or diversity injection.

7.4 Low MAP@10 is a Dataset Artefact, Not a Model Failure

MAP@10 values in the range of 0.003–0.0085 appear very low in absolute terms, but this is consistent with the literature on explicit feedback evaluation. The Netflix Prize test set contains only movies users actually chose to rate — it is not a randomly drawn held-out set. A model that recommends an excellent but never-rated movie receives no credit. This is known as the missing data problem in information retrieval evaluation, and is the primary reason MAP@10 appears low even for strong models.

8. Future Improvements

- Temporal modelling: Incorporate rating timestamps to model evolving user preferences (time-weighted SVD, sequential attention models).
- Content-based features: Add movie metadata (genre, director, cast) to enable hybrid content-CF filtering and address cold-start for new users.
- Exposure-aware evaluation: Use held-out positive/negative pairs to obtain unbiased ranking metrics (e.g., sampled softmax evaluation as in NCF paper).
- Diversity re-ranking: Apply maximal marginal relevance (MMR) or determinantal point processes to inject diversity into final recommendation lists without sacrificing per-user accuracy.
- Deployment: The Streamlit dashboard (app/streamlit_app.py) demonstrates a production-ready user interface. A natural next step is wrapping the recommendation engine in a FastAPI service with caching, enabling real-time recommendations at sub-100ms latency.
- Neural architecture improvements: Experiment with graph-based approaches (LightGCN), autoencoders (EASE, MultiVAE), or transformer-based sequential models (BERT4Rec) — all of which have shown strong Netflix Prize benchmark results in recent literature.

Repository: github.com/25f1001872/netflix-recsys

All models, code, and pre-trained artifacts are available for download and reproduction.